

5	Interlude: Process API	37
5.1	The <code>fork()</code> System Call	37
5.2	The <code>wait()</code> System Call	39
5.3	Finally, The <code>exec()</code> System Call	40
5.4	Why? Motivating The API	41
5.5	Other Parts Of The API	44
5.6	Summary	44
	References	45
	Homework (Code)	46
6	Mechanism: Limited Direct Execution	49
6.1	Basic Technique: Limited Direct Execution	49
6.2	Problem #1: Restricted Operations	50
6.3	Problem #2: Switching Between Processes	54
6.4	Worried About Concurrency?	58
6.5	Summary	59
	References	61
	Homework (Measurement)	62
7	Scheduling: Introduction	63
7.1	Workload Assumptions	63
7.2	Scheduling Metrics	64
7.3	First In, First Out (FIFO)	64
7.4	Shortest Job First (SJF)	66
7.5	Shortest Time-to-Completion First (STCF)	67
7.6	A New Metric: Response Time	68
7.7	Round Robin	69
7.8	Incorporating I/O	71
7.9	No More Oracle	72
7.10	Summary	72
	References	73
	Homework	74
8	Scheduling:	
	The Multi-Level Feedback Queue	75
8.1	MLFQ: Basic Rules	76
8.2	Attempt #1: How To Change Priority	77
8.3	Attempt #2: The Priority Boost	80
8.4	Attempt #3: Better Accounting	81
8.5	Tuning MLFQ And Other Issues	82
8.6	MLFQ: Summary	83
	References	85
	Homework	86
9	Scheduling: Proportional Share	87
9.1	Basic Concept: Tickets Represent Your Share	87
9.2	Ticket Mechanisms	89

9.3	Implementation	90
9.4	An Example	91
9.5	How To Assign Tickets?	92
9.6	Why Not Deterministic?	92
9.7	Summary	93
	References	95
	Homework	96
10	Multiprocessor Scheduling (Advanced)	97
10.1	Background: Multiprocessor Architecture	98
10.2	Don't Forget Synchronization	100
10.3	One Final Issue: Cache Affinity	101
10.4	Single-Queue Scheduling	101
10.5	Multi-Queue Scheduling	103
10.6	Linux Multiprocessor Schedulers	106
10.7	Summary	106
	References	107
11	Summary Dialogue on CPU Virtualization	109
12	A Dialogue on Memory Virtualization	111
13	The Abstraction: Address Spaces	113
13.1	Early Systems	113
13.2	Multiprogramming and Time Sharing	114
13.3	The Address Space	115
13.4	Goals	117
13.5	Summary	119
	References	120
14	Interlude: Memory API	123
14.1	Types of Memory	123
14.2	The <code>malloc()</code> Call	124
14.3	The <code>free()</code> Call	126
14.4	Common Errors	126
14.5	Underlying OS Support	129
14.6	Other Calls	130
14.7	Summary	130
	References	131
	Homework (Code)	132
15	Mechanism: Address Translation	135
15.1	Assumptions	136
15.2	An Example	136
15.3	Dynamic (Hardware-based) Relocation	139
15.4	Hardware Support: A Summary	142
15.5	Operating System Issues	143

15.6 Summary	146
References	147
Homework	148
16 Segmentation	149
16.1 Segmentation: Generalized Base/Bounds	149
16.2 Which Segment Are We Referring To?	152
16.3 What About The Stack?	153
16.4 Support for Sharing	154
16.5 Fine-grained vs. Coarse-grained Segmentation	155
16.6 OS Support	155
16.7 Summary	157
References	158
Homework	160
17 Free-Space Management	161
17.1 Assumptions	162
17.2 Low-level Mechanisms	163
17.3 Basic Strategies	171
17.4 Other Approaches	173
17.5 Summary	175
References	176
Homework	177
18 Paging: Introduction	179
18.1 A Simple Example And Overview	179
18.2 Where Are Page Tables Stored?	183
18.3 What's Actually In The Page Table?	184
18.4 Paging: Also Too Slow	185
18.5 A Memory Trace	186
18.6 Summary	189
References	190
Homework	191
19 Paging: Faster Translations (TLBs)	193
19.1 TLB Basic Algorithm	193
19.2 Example: Accessing An Array	195
19.3 Who Handles The TLB Miss?	197
19.4 TLB Contents: What's In There?	199
19.5 TLB Issue: Context Switches	200
19.6 Issue: Replacement Policy	202
19.7 A Real TLB Entry	203
19.8 Summary	204
References	205
Homework (Measurement)	207
20 Paging: Smaller Tables	211

20.1	Simple Solution: Bigger Pages	211
20.2	Hybrid Approach: Paging and Segments	212
20.3	Multi-level Page Tables	215
20.4	Inverted Page Tables	222
20.5	Swapping the Page Tables to Disk	223
20.6	Summary	223
	References	224
	Homework	225
21	Beyond Physical Memory: Mechanisms	227
21.1	Swap Space	228
21.2	The Present Bit	229
21.3	The Page Fault	230
21.4	What If Memory Is Full?	231
21.5	Page Fault Control Flow	232
21.6	When Replacements Really Occur	233
21.7	Summary	234
	References	235
22	Beyond Physical Memory: Policies	237
22.1	Cache Management	237
22.2	The Optimal Replacement Policy	238
22.3	A Simple Policy: FIFO	240
22.4	Another Simple Policy: Random	242
22.5	Using History: LRU	243
22.6	Workload Examples	244
22.7	Implementing Historical Algorithms	247
22.8	Approximating LRU	248
22.9	Considering Dirty Pages	249
22.10	Other VM Policies	250
22.11	Thrashing	250
22.12	Summary	251
	References	252
	Homework	254
23	The VAX/VMS Virtual Memory System	255
23.1	Background	255
23.2	Memory Management Hardware	256
23.3	A Real Address Space	257
23.4	Page Replacement	259
23.5	Other Neat VM Tricks	260
23.6	Summary	262
	References	263
24	Summary Dialogue on Memory Virtualization	265

II Concurrency	269
25 A Dialogue on Concurrency	271
26 Concurrency: An Introduction	273
26.1 An Example: Thread Creation	274
26.2 Why It Gets Worse: Shared Data	277
26.3 The Heart Of The Problem: Uncontrolled Scheduling . . .	279
26.4 The Wish For Atomicity	281
26.5 One More Problem: Waiting For Another	283
26.6 Summary: Why in OS Class?	283
References	285
Homework	286
27 Interlude: Thread API	289
27.1 Thread Creation	289
27.2 Thread Completion	290
27.3 Locks	293
27.4 Condition Variables	295
27.5 Compiling and Running	297
27.6 Summary	297
References	299
28 Locks	301
28.1 Locks: The Basic Idea	301
28.2 Pthread Locks	302
28.3 Building A Lock	303
28.4 Evaluating Locks	303
28.5 Controlling Interrupts	304
28.6 Test And Set (Atomic Exchange)	306
28.7 Building A Working Spin Lock	307
28.8 Evaluating Spin Locks	309
28.9 Compare-And-Swap	309
28.10 Load-Linked and Store-Conditional	311
28.11 Fetch-And-Add	312
28.12 Too Much Spinning: What Now?	313
28.13 A Simple Approach: Just Yield, Baby	314
28.14 Using Queues: Sleeping Instead Of Spinning	315
28.15 Different OS, Different Support	317
28.16 Two-Phase Locks	318
28.17 Summary	319
References	320
Homework	322
29 Lock-based Concurrent Data Structures	325
29.1 Concurrent Counters	325
29.2 Concurrent Linked Lists	330

29.3	Concurrent Queues	333
29.4	Concurrent Hash Table	334
29.5	Summary	336
	References	337
30	Condition Variables	339
30.1	Definition and Routines	340
30.2	The Producer/Consumer (Bounded Buffer) Problem	343
30.3	Covering Conditions	351
30.4	Summary	352
	References	353
31	Semaphores	355
31.1	Semaphores: A Definition	355
31.2	Binary Semaphores (Locks)	357
31.3	Semaphores As Condition Variables	358
31.4	The Producer/Consumer (Bounded Buffer) Problem	360
31.5	Reader-Writer Locks	364
31.6	The Dining Philosophers	366
31.7	How To Implement Semaphores	369
31.8	Summary	370
	References	371
32	Common Concurrency Problems	373
32.1	What Types Of Bugs Exist?	373
32.2	Non-Deadlock Bugs	374
32.3	Deadlock Bugs	377
32.4	Summary	385
	References	386
33	Event-based Concurrency (Advanced)	389
33.1	The Basic Idea: An Event Loop	389
33.2	An Important API: <code>select()</code> (or <code>poll()</code>)	390
33.3	Using <code>select()</code>	391
33.4	Why Simpler? No Locks Needed	392
33.5	A Problem: Blocking System Calls	393
33.6	A Solution: Asynchronous I/O	393
33.7	Another Problem: State Management	396
33.8	What Is Still Difficult With Events	397
33.9	Summary	397
	References	398
34	Summary Dialogue on Concurrency	399